Gestion des livraisons

TP Algorithmique — Rapport des choix algorithmiques

Glenn Guillard

22 octobre 2026

Langage : Python 3

1. Introduction

Une entreprise de livraison doit gérer une collection de colis (identifiant, poids, valeur, priorité, destination) pouvant atteindre 100 000 éléments, puis préparer chaque jour le chargement d'un camion de capacité limitée. Le programme permet d'ajouter, afficher, rechercher et trier les colis, de calculer un chargement de valeur maximale, d'y intégrer la priorité, et de mesurer les performances.

Choix du langage : Python

- **Lisibilité** : le code reste court, ce qui permet de se concentrer sur les algorithmes plutôt que sur la gestion mémoire.
- **Algorithmes codés à la main** : tri fusion, tri par comptage et sac à dos sont implémentés sans fonction native. Le tri natif `sorted()` n'est utilisé que comme point de comparaison dans les mesures.
- **Bibliothèque standard uniquement** : aucune dépendance à installer.
- **Contrepartie** : Python est plus lent que le C. Les mesures (partie 7) montrent que les volumes demandés restent largement traitables.

Organisation du code

Fichier	Rôle
main.py	Point d'entrée : charge les 5 colis de l'énoncé et lance le menu
colis.py	Classe Colis et GestionnaireColis (liste + dictionnaire)
tri.py	Tri fusion, tri par comptage, tri par insertion (comparaison)
sac_a_dos.py	Chargement : programmation dynamique, glouton, force brute
priorite.py	Score combinant valeur et priorité
menu.py	Interface console
benchmark.py	Génération de colis aléatoires et mesures de temps

Lancement : `python3 src/main.py`. Le menu propose les fonctions de l'énoncé, plus la génération de colis aléatoires (option 8) et les mesures de performances (option 9).

2. Choix n°1 — Stockage et recherche par identifiant

Les colis sont stockés dans **deux structures complémentaires** qui pointent vers les mêmes objets (aucune copie) : une **liste** qui conserve l'ordre d'ajout, utilisée pour l'affichage et les tris, et un **dictionnaire** `id → colis`, utilisé pour la recherche. Le dictionnaire sert aussi à refuser les identifiants en double lors de l'ajout.

Problème	Retrouver rapidement un colis à partir de son identifiant, parmi jusqu'à 100 000 colis, opération utilisée très souvent.
Solution	Table de hachage : le dictionnaire Python associe chaque identifiant à son colis.
Temps	Recherche et ajout en **O(1) en moyenne**, O(n) dans le pire cas (toutes les clés en collision).
Espace	**O(n)** : une entrée par colis, plus des cases vides pour garder la table peu remplie.
Alternative	Parcours de la liste : O(n). Liste triée + recherche dichotomique : O(log n) mais l'insertion coûte O(n) (décalage).
Justification	La recherche est l'opération la plus fréquente. Le temps reste quasi constant quand n augmente (mesuré : 0,1 ms à 0,5 ms pour 1 000 recherches, contre 1,3 s en linéaire à 100 000 colis).

Fonctionnement interne du dictionnaire

- **Hachage** : la clé est transformée en entier par `hash()` (pour un entier, c'est l'entier lui-même). Les bits de poids faible de ce hash donnent la case de départ dans un tableau dont la taille est une puissance de 2.
- **Collisions** : CPython utilise l'adressage ouvert. Si la case est occupée par une autre clé, il sonde d'autres cases selon une suite pseudo-aléatoire qui dépend du hash complet, jusqu'à trouver la clé ou une case vide.
- **Redimensionnement** : quand la table est remplie aux 2/3, elle est agrandie et toutes les clés sont réinsérées. Cette opération coûte O(n) mais reste rare : le coût moyen d'un ajout est O(1) (coût amorti).
- Comme la table n'est jamais pleine, la recherche examine en moyenne un nombre constant de cases, d'où le O(1).

La liste Python fonctionne de façon similaire : c'est un tableau dynamique qui réserve de la place en avance, donc `append()` est en O(1) amorti.

3. Choix n°2 — Tri par poids et par valeur

Problème	Afficher les colis triés par poids (croissant) ou par valeur (décroissante), sans modifier l'ordre d'origine.
Solution	Tri fusion itératif (« de bas en haut ») codé à la main, sur une copie de la liste.
Temps	**O(n log n)** dans tous les cas : $\log_2 n$ passes, chacune de coût O(n).
Espace	**O(n)** : un tableau auxiliaire de même taille.
Alternative	Tri par insertion : $O(n^2)$, simple mais inutilisable à 100 000 colis. Tri rapide : O(n log n) en moyenne mais $O(n^2)$ dans le pire cas et non stable. `sorted()` (Timsort, hybride fusion + insertion) : O(n log n).
Justification	Complexité garantie quel que soit l'ordre des données, et tri **stable** : deux colis de même poids gardent leur ordre relatif. Contrairement à `sorted()`, son fonctionnement est entièrement maîtrisé et expliqué.

Principe

On fusionne d'abord des blocs de taille 1 deux à deux, puis des blocs de taille 2, 4, 8... jusqu'à n. Fusionner deux blocs triés revient à comparer leurs premiers éléments et à recopier le plus petit, ce qui coûte un temps proportionnel à leur taille. À égalité, on prend

l'élément de gauche, ce qui garantit la stabilité. La version itérative évite la récursion (pas de limite de profondeur en Python).

Optimisation : la clé de tri (poids ou valeur) est calculée **une seule fois** par colis au départ, et stockée avec lui dans un couple `(clé, colis)`. Pour l'ordre décroissant, on trie simplement sur l'opposé de la clé.

Mesures : à 100 000 colis, le tri fusion prend environ 280 ms, contre 16 ms pour `sorted()`. L'écart (≈ facteur 18) ne vient pas de l'algorithme, de même complexité, mais du fait que Timsort est écrit en C alors que notre tri est interprété. Les deux courbes ont la même pente.

4. Choix n°3 — Tri par priorité

Problème	Afficher les colis par priorité décroissante. La priorité ne prend que k = 5 valeurs (1 à 5).
Solution	Tri par comptage (version « seaux ») : un seau par niveau de priorité, on y range chaque colis, puis on concatène les seaux de la priorité 5 à la priorité 1.
Temps	**O(n + k)**, soit O(n) puisque k = 5 est constant.
Espace	**O(n + k)** : les seaux contiennent au total les n colis.
Alternative	Réutiliser le tri fusion : O(n log n). Ce serait correct, mais moins efficace.
Justification	Le tri par comptage ne compare jamais deux colis : il exploite le fait que les valeurs possibles sont peu nombreuses. Il est linéaire et stable. Mesuré : 27 ms à 100 000 colis, environ 10 fois plus rapide que le tri fusion.

Limite : cette méthode n'est intéressante que si k reste petit devant n. Elle ne conviendrait pas pour le poids ou la valeur, dont les valeurs possibles sont beaucoup plus nombreuses.

5. Choix n°4 — Chargement du camion

Il faut choisir un sous-ensemble de colis dont le poids total ne dépasse pas la capacité W, en maximisant la valeur totale. Chaque colis est pris ou laissé : c'est le **problème du sac à dos 0/1**.

Problème	Trouver le chargement de valeur maximale respectant la capacité du camion (pas seulement un chargement qui rentre).
Solution	Programmation dynamique, avec reconstruction de la liste des colis chargés.
Temps	**O(n × W)** : pour chaque colis, on examine chaque capacité de 0 à W.
Espace	**O(W)** entiers pour les meilleures valeurs, plus **n × W octets** pour mémoriser les choix (reconstruction).
Alternative	Force brute : $O(2^n)$, optimale mais impossible au-delà d'environ 25 colis. Glouton par ratio valeur/poids : O(n log n), rapide mais pas toujours optimal.
Justification	C'est la seule des trois approches à la fois **optimale** et utilisable sur de grands volumes, tant que la capacité reste raisonnable (les poids sont des entiers en kg).

Principe

On note `meilleur[w]` la meilleure valeur atteignable avec une capacité w en n'utilisant que les colis déjà examinés. On examine les colis un par un. Pour un colis de poids p et de valeur v :

```
meilleur[w] = max( meilleur[w],            # on ne prend pas le colis
                   meilleur[w - p] + v )    # on le prend (si w >= p)
```

La nouvelle ligne est calculée à partir de l'ancienne, donc chaque colis ne peut être pris qu'une seule fois. Pour chaque colis, on garde une ligne d'octets indiquant, pour chaque capacité, s'il a été pris. On retrouve ensuite la solution en remontant du dernier colis au premier, en partant de la capacité W : si le colis a été pris, on l'ajoute et on retire son poids. Les colis plus lourds que W ou de valeur nulle sont écartés dès le départ.

Réponses aux questions de l'énoncé

- **Pourquoi cette stratégie ?** L'objectif est la meilleure valeur totale, pas une solution acceptable. La programmation dynamique garantit l'optimum sans tester les 2^n combinaisons.
- **La solution est-elle toujours optimale ?** Oui. Chaque case est le maximum des deux seules possibilités (prendre ou non le colis), donc l'optimum se construit à partir des optimums des sous-problèmes. Le programme le vérifie : pour 15 colis ou moins, il compare son résultat à la force brute, et 500 instances aléatoires ont donné le même résultat.
- **Complexité temporelle :** O(n × W). Elle est dite pseudo-polynomiale : elle dépend de la valeur de W, pas seulement du nombre de colis.
- **Complexité spatiale :** O(W) pour les valeurs, plus n × W octets pour la reconstruction. Pour 100 000 colis et W = 200 kg, cela représente environ 20 Mo.
- **Une autre approche ?** Oui, le glouton (voir ci-dessous) ou la force brute, et pour de très grandes capacités une méthode de séparation et évaluation (branch and bound).

Contre-exemple : le glouton n'est pas optimal

Avec les colis de l'énoncé et une capacité de **9 kg** :

Colis	Poids	Valeur	Valeur / poids
A	8	20	2,50
B	6	25	4,17
C	5	18	3,60
D	4	14	3,50

- **Glouton** : il prend B (meilleur ratio, 6 kg). Il reste 3 kg, aucun autre colis ne rentre. Valeur : **25**.
- **Programmation dynamique** : elle choisit C + D (9 kg). Valeur : **32**, soit 28 % de plus.

Le glouton se fait piéger parce qu'un bon ratio ne garantit pas une bonne utilisation de la place restante.

6. Choix n°5 — Prise en compte de la priorité

L'énoncé n'impose pas de formule. La règle retenue remplace la valeur par un **score**, que le même algorithme de sac à dos maximise :

```
score = valeur × (10 + alpha × (priorité − 1))      priorité : 1 (faible) à 5 (forte)
```

Problème	Favoriser les colis prioritaires lors du chargement, en tenant compte du poids, de la valeur et de la priorité.
Solution	Calcul d'un score par colis, puis sac à dos en programmation dynamique sur ce score.
Temps	O(n) pour les scores, puis O(n × W) pour le chargement.
Espace	O(n) pour les scores, en plus de celui du sac à dos.
Alternative	Charger d'abord tous les colis de priorité maximale, puis optimiser la valeur avec la place restante. Ou trier par priorité puis par ratio (glouton).
Justification	Les trois critères ont chacun un rôle clair et le résultat reste optimal pour le score choisi.

Justification

- **Poids** : il reste la **contrainte** du camion. On ne l'intègre pas au score, car le sac à dos fait déjà l'arbitrage entre ce qu'un colis rapporte et la place qu'il prend.
- **Valeur** : elle reste la base du score, donc un colis prioritaire sans valeur n'est pas favorisé.
- **Priorité** : elle **multiplie** la valeur. Avec alpha = 5, un colis de priorité 5 compte 3 fois sa valeur, un colis de priorité 1 une seule fois.
- **Paramètre alpha réglable** : alpha = 0 revient au cas sans priorité. Plus alpha est grand, plus la priorité pèse. L'entreprise peut ainsi ajuster le compromis.
- **Score entier** : il permet de réutiliser exactement le même algorithme optimal.
- Par rapport à l'alternative « prioritaires d'abord », la règle évite de remplir le camion avec un colis prioritaire très lourd et presque sans valeur.

Exemple (colis A à D de l'énoncé, capacité 12 kg, alpha = 5)

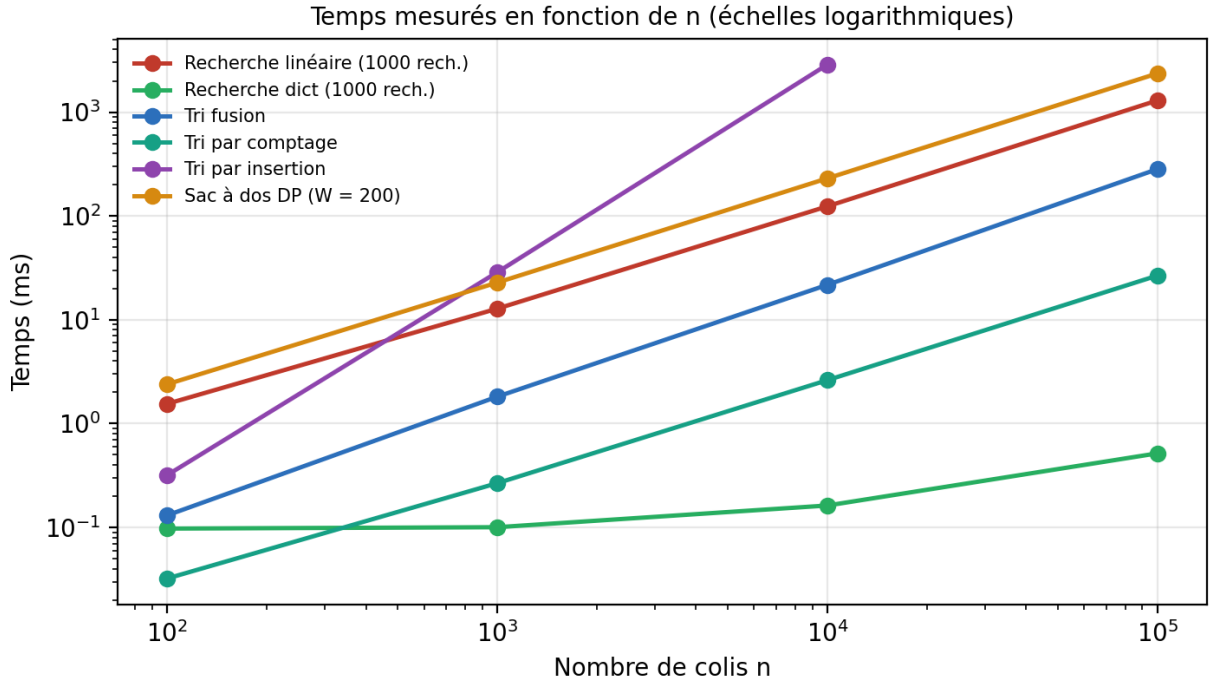
Critère	Colis chargés	Poids	Valeur	Priorités
Valeur seule	B + C	11 kg	43	1 et 4
Valeur + priorité	A + D	12 kg	34	5 et 3

Avec la priorité, B (priorité 1) est écarté au profit de A (priorité 5) : on accepte une valeur un peu plus faible pour livrer les colis les plus urgents. C'est exactement l'effet recherché.

7. Analyse des performances

Les colis sont générés aléatoirement (poids 1 à 50 kg, valeur 1 à 100, priorité 1 à 5, graine fixe pour la reproductibilité). Les recherches portent sur 1 000 identifiants existants tirés au hasard. Le sac à dos est mesuré avec une capacité de 200 kg. Le tri par insertion n'est pas lancé au-delà de 10 000 colis, car il serait trop long. Temps en millisecondes, mesurés avec `time.perf_counter()` (option 9 du menu).

n	Rech. linéaire	Rech. dict	Tri fusion	sorted()	Tri comptage	Tri insertion	Sac à dos DP
100	1,552	0,098	0,131	0,016	0,032	0,319	2,394
1 000	12,8	0,101	1,817	0,157	0,267	28,7	22,8
10 000	123,7	0,163	21,7	1,530	2,622	2 867,8	230,1
100 000	1 296,3	0,518	282,6	15,6	26,7	–	2 358,3



Sur un graphique log-log, la pente d'une courbe donne l'exposant : une pente de 1 correspond à O(n), une pente de 2 à O(n²), une courbe plate à O(1).

Opération 1 : la recherche d'un colis

- **Complexité** : O(n) par recherche avec un parcours, O(1) en moyenne avec le dictionnaire.
- **Évolution** : le temps de la recherche linéaire est multiplié par environ 10 à chaque fois que n est multiplié par 10 (1,6 ms → 1 296 ms). Celui du dictionnaire reste quasi constant (0,1 ms → 0,5 ms, la légère hausse étant due aux accès mémoire sur une table plus grande).
- **Le choix reste-t-il adapté ?** Oui. À 100 000 colis, le dictionnaire est environ 2 500 fois plus rapide. Comme l'entreprise recherche régulièrement des colis, c'était l'opération à optimiser en priorité.
- **Amélioration** : aucune nécessaire. Pour rechercher par destination, on pourrait ajouter un second dictionnaire `destination → liste de colis`.

Opération 2 : le chargement du camion

- **Complexité** : O(n × W) en temps, n × W octets en mémoire.
- **Évolution** : à capacité fixe, le temps est proportionnel à n (2,4 ms → 2,4 s). À n fixe, il est proportionnel à W : pour 10 000 colis, 107 ms à 100 kg, 228 ms à 200 kg, 474 ms à 400 kg, 949 ms à 800 kg.

- **Le choix reste-t-il adapté ?** Oui pour un camion réel (quelques centaines à quelques milliers de kg) et quelques milliers de colis. Il devient coûteux pour 100 000 colis avec une grande capacité : avec W = 10 000 kg, la table des choix atteindrait 1 Go. Le programme refuse alors le calcul au-delà d'une limite mémoire au lieu de planter.
- **Améliorations** : (1) présélectionner les colis : un camion de 200 kg ne contient au mieux que quelques dizaines de colis, on peut donc ne garder que les meilleurs ratios avant de lancer le calcul ; (2) réduire la précision des poids (par paliers de 5 kg) pour diminuer W ; (3) pour les très grandes instances, utiliser le glouton, rapide et généralement proche de l'optimum, en acceptant de perdre la garantie.

Opération 3 : les tris

- **Complexité** : O(n log n) pour le tri fusion, O(n) pour le tri par comptage, $O(n^2)$ pour le tri par insertion.
- **Évolution** : quand n est multiplié par 10, le tri par insertion est multiplié par environ 100 (0,3 ms → 2,9 s entre 100 et 10 000 colis) : il prendrait plusieurs minutes à 100 000. Le tri fusion est multiplié par environ 12, ce qui correspond au facteur 10 × (log 10n / log n).
- **Le choix reste-t-il adapté ?** Oui : moins de 0,3 s pour trier 100 000 colis.
- **Amélioration** : utiliser `sorted()` (Timsort, même complexité mais implémenté en C, 18 fois plus rapide), ou garder les listes triées en mémoire si les tris sont plus fréquents que les ajouts.

8. Conclusion

Les structures et algorithmes retenus tiennent la charge demandée : à 100 000 colis, la recherche reste instantanée, les tris durent moins d'un tiers de seconde et le chargement est calculé de façon optimale. Les mesures confirment les complexités théoriques, et la comparaison avec les alternatives (parcours linéaire, tri par insertion, glouton) montre l'intérêt de chaque choix.

La principale limite est le coût du sac à dos, qui dépend de la capacité du camion. Pour aller plus loin, on pourrait présélectionner les colis avant le calcul, gérer plusieurs camions, ou sauvegarder les colis dans un fichier pour les retrouver d'une exécution à l'autre.